HEXLOGIC

WEB APPLICATION PENETRATION TESTING

“demo.testfire.net”

Document Information

Project Name	Web Application Penetration Testing
Document Title	Web Application Penetration Test Report
Document Type	Final Report
Classification	Confidential & Restricted
Created By	HexLogic Offensive Security

Document History

Version	Author	Date	Change Description
1.0	HexLogic	30 May 2025	Final Report

Table of Contents

1 Introduction

This report describes the proceedings and results of the web application penetration test conducted by HexLogic against demo.testfire.net. It enlists the findings identified during the engagement, their business impact and recommended remediation.

1.1 Objective

The objective of the assessment was to evaluate the security posture of demo.testfire.net and to provide a final security review report comprising a remediation strategy and recommendations to help mitigate the identified vulnerabilities and risks. The assessment was conducted presuming the malicious intent of an external adversary in order to identify associated business risk and its impact.

1.2 Assessment Duration

Test Duration	15 May 2025 – 23 May 2025
Total Days	7 Days
Note	First Test

1.3 Team Contact Details

Name	HexLogic
Contact	security@hexlogic.io
Role	Penetration Testing Team

2 Scope

This section defines the scope and boundaries of the engagement.

2.1 Target Scope

- https://demo.testfire.net/

2.2 Assessment Attributes

Starting Vector	External
Target Criticality	Assessment Environment
Assessment Nature	Cautious & calculated
Assessment Conspicuity	Clear
Proof of Concept(s)	Attached wherever possible and applicable

2.3 Assessment Type and Methods

The assessment was performed in a grey-box manner, primarily manual in nature while using a limited, necessary tool-set to make the process more efficient, preserving the imitation of a real-world adversary. Testing covered authentication, session management, authorization, input handling and injection, business logic and configuration review.

2.4 Methodology & Risk Classification

The following risk classification is used throughout this report. Findings are rated using CVSS v3.1 and business context.

Critical	Trivially exploitable issues leading to full system or data compromise; remediate immediately.
High	Serious issues allowing significant unauthorised access or impact; remediate as a priority.
Medium	Issues exploitable under certain conditions, often combined with others; schedule remediation.
Low	Limited-impact weaknesses and defence-in-depth gaps; address during routine hardening.
Info	No direct threat, but may reveal sensitive information useful to an attacker.

3 Executive Summary

The target in scope was reviewed for the adequacy of its existing security controls in accordance with the OWASP Web Security Testing Guide (WSTG) and OSSTMM and industry best practices. The aim of the testing was to establish whether the target could be compromised to allow unauthorised access to sensitive data or systems.

A total of 10 vulnerabilities were identified during the assessment, comprising 2 Critical, 3 High, 4 Medium and 1 Low-risk findings. The most serious issues are SQL injection flaws that permit authentication bypass and full database extraction; these allow complete compromise of the application and its data and must be remediated immediately, followed by the high-severity cross-site scripting, authorization and authentication weaknesses.

3.1 Graphical Representation of Vulnerabilities

The table below summarises the overall risk identified during the assessment.

Critical	High	Medium	Low	Total
2	3	4	1	10

4 Compliance – Detailed Summary

The final risk value of each vulnerability is derived by considering the likelihood of exploitation and the impact on the business of the assessed target.

No	Findings	Severity	CVSS 3.1	Status
1	SQL Injection — Authentication Bypass	Critical	9.8	Open
2	SQL Injection — Database Extraction (UNION-based)	Critical	9.1	Open
3	Reflected Cross-Site Scripting in Search	High	8.2	Open
4	Insecure Direct Object Reference — Account Access	High	7.7	Open
5	Broken Authentication — No Account Lockout or Rate Limiting	High	7.3	Open
6	Cross-Site Request Forgery — Funds Transfer	Medium	6.5	Open
7	Outdated JavaScript Library with Known Vulnerabilities	Medium	6.2	Open
8	Verbose Error Messages & Stack-Trace Disclosure	Medium	5.3	Open
9	Missing HTTP Security Headers	Medium	5.3	Open
10	Username Enumeration via Login Responses	Low	3.7	Open

5 Detailed Technical Summary

This section provides the technical detail for each finding, including a comprehensive description, business impact, reproduction steps, proof of concept and remediation guidance.

5.1 SQL Injection — Authentication Bypass

Critical	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	9.8	Open

Description

The application's login function builds its SQL query by directly concatenating the user-supplied username and password into the statement, without using parameterised queries or prepared statements. As a result, SQL metacharacters submitted in the credentials are interpreted as part of the query logic rather than as data. By submitting the username admin'-- the attacker terminates the username string and comments out the remainder of the query, including the password comparison, so the database returns the administrator record and the application grants a fully authenticated session.

Because the same code path is reachable without any prior authentication, the flaw is exploitable by any anonymous user on the internet and requires no special tooling beyond an HTTP client. The injection point can be extended beyond authentication bypass to read, modify or delete arbitrary data in the underlying database.

Vulnerable URL

https://demo.testfire.net/login.jsp

Impact

- Complete authentication bypass — an attacker can log in as any user, including administrators, without knowing a password.
- Full, unauthorised access to every customer account, balance, statement and item of personal data held by the application.
- The injectable parameter can be weaponised for blind or UNION-based extraction of the entire database, and in some configurations for command execution on the database host.
- A breach of this nature is reportable under data-protection regimes such as GDPR and would cause significant regulatory, financial and reputational damage.

Steps to Reproduce

1. Browse to the login page and intercept the POST request to /doLogin in an interception proxy.
2. Replace the uid parameter value with admin'-- and set passw to any value.
3. Forward the modified request to the server.
4. Observe a 302 redirect to /bank/main.jsp accompanied by a valid authenticated session — the attacker is logged in as admin.

Proof of Concept

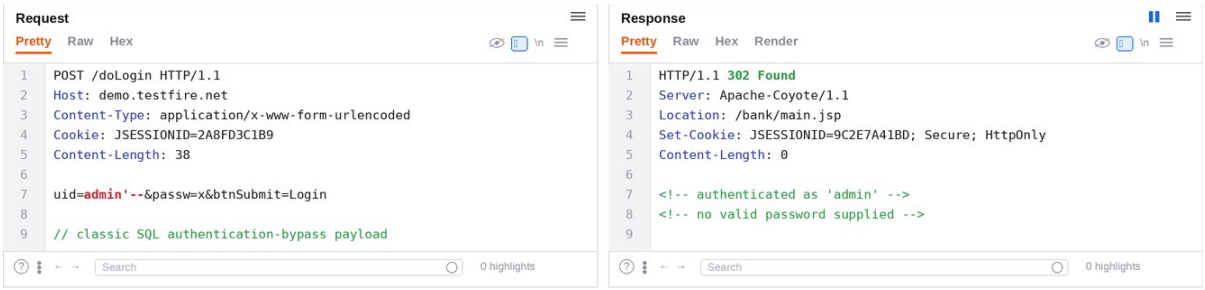

Figure 1. Burp Repeater — the payload admin'-- bypasses authentication and the server returns a logged-in session.

Recommendations

- Use parameterised queries / prepared statements for every database interaction so that user input is always treated as data and never as executable SQL.
- Adopt an ORM or query builder that parameterises by default and review all existing queries for unsafe string concatenation.
- Apply strict server-side input validation (type, length, format) as a defence-in-depth control.
- Run database connections under a least-privilege account that cannot read credential tables or execute administrative commands.
- Add automated SQL-injection regression tests to the CI pipeline and consider a Web Application Firewall as a compensating control during remediation.

References

- OWASP — SQL Injection Prevention Cheat Sheet
- PortSwigger — SQL injection
- CWE-89

5.2 SQL Injection — Database Extraction (UNION-based)

Critical	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N	9.1	Open

Description

The account-listing parameter is concatenated unsafely into a SQL query, allowing UNION-based SQL injection. By appending a crafted UNION SELECT statement an attacker can append the results of an arbitrary query to the application's normal output, returning columns from any table the database account can read — including the application's user and credential store.

Because the response renders the injected rows directly, the attacker can iterate through tables and columns to exfiltrate the entire database contents. Credentials recovered through this flaw enable direct, persistent account takeover that survives any password reset of the originally compromised account.

Vulnerable URL

https://demo.testfire.net/bank/showAccount?listAccounts=

Impact

- Disclosure of the complete database, including usernames, password hashes or plaintext passwords, and customer financial data.
- Credentials recovered here provide durable, legitimate-looking access that is difficult to detect.
- Depending on database privileges, the same flaw permits data modification, deletion and potentially code execution on the database server.

Steps to Reproduce

1. Authenticate and capture a request to /bank/showAccount?listAccounts=.
2. Append the payload 800000 UNION SELECT username,password,3 FROM users-- to the parameter.
3. Forward the request and observe rows from the users table rendered in the response body.

Proof of Concept

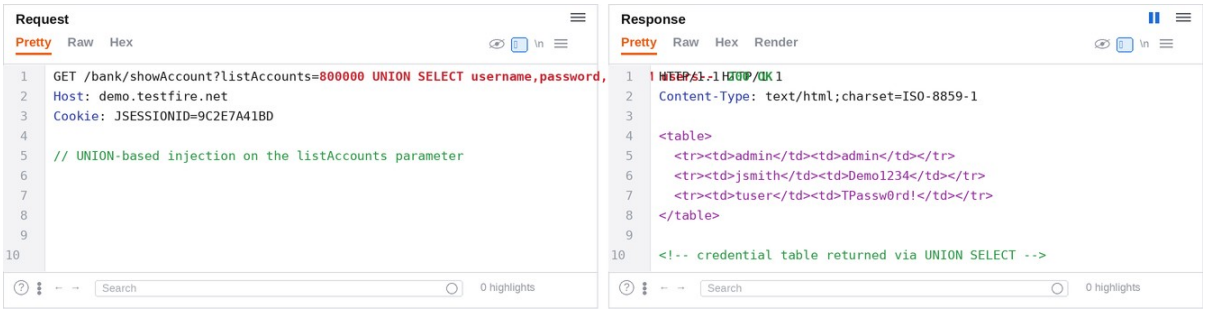

Figure 2. Burp Repeater — a UNION SELECT injected into listAccounts returns the application's username and password table.

Recommendations

- Parameterise all database queries and validate that numeric parameters are strictly integers before use.

- Store passwords using a strong, salted, adaptive hash such as bcrypt or Argon2 so that a database leak does not expose usable plaintext credentials.
- Apply least-privilege database accounts so the web tier cannot read credential or administrative tables.
- Implement generic error handling so the database structure is not revealed to attackers.
- Deploy a Web Application Firewall as a temporary compensating control and add injection tests to CI.

References

- OWASP — SQL Injection Prevention Cheat Sheet
- CWE-89

5.3 Reflected Cross-Site Scripting in Search

High	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N	8.2	Open

Description

The search feature reflects the user-supplied query parameter back into the HTML response without performing any output encoding or contextual escaping. An attacker can therefore craft a URL containing a script payload that, when opened by a victim, is parsed and executed by the victim's browser within the security context (origin) of the application.

Reflected XSS is typically delivered through a malicious link sent by email, chat or a third-party site. When the victim is an authenticated user, the attacker's script runs with the victim's session and can perform any action the victim can, or silently exfiltrate session material to an attacker-controlled server.

Vulnerable URL

https://demo.testfire.net/search.jsp?query=

Impact

- Execution of attacker-controlled JavaScript in the victim's authenticated session.
- Theft of session tokens and anti-CSRF tokens, leading to session hijacking and account takeover.
- Credential harvesting via injected fake login forms, and arbitrary actions performed as the victim.
- When targeted at an administrator, the impact escalates to control of administrative functionality (reflected in the scope change).

Steps to Reproduce

1. Request /search.jsp?query=<script>alert(document.cookie)</script>.
2. Observe the payload reflected, unencoded, within the response body.
3. Load the URL in a browser and confirm the script executes, as demonstrated by the alert dialog showing the session cookie.

Proof of Concept

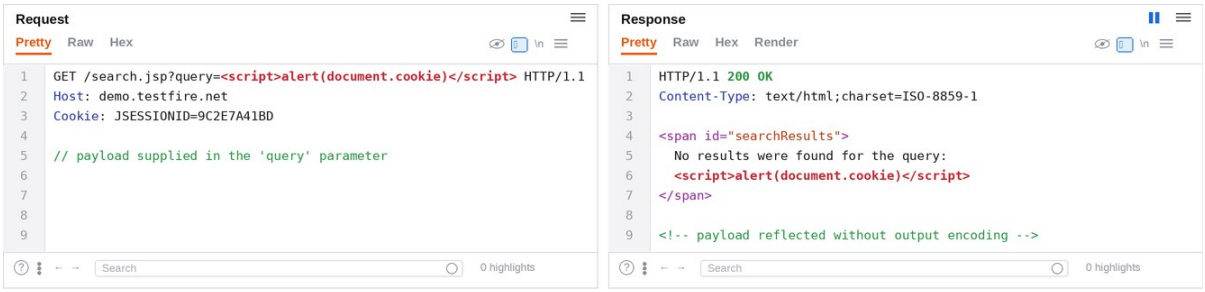

Figure 3. Burp Repeater — the script payload is reflected unencoded and executes in the browser, leaking document.cookie.

Recommendations

- Apply context-aware output encoding to every piece of user-controlled data rendered into a response (HTML, attribute, JavaScript and URL contexts).

- Deploy a strict Content-Security-Policy to limit the impact of any residual injection.
- Set the HttpOnly attribute on session cookies so that script cannot read them.
- Validate and normalise input, and prefer a templating framework that auto-escapes by default.
- Add automated XSS tests covering reflected, stored and DOM-based sinks.

References

- OWASP — Cross Site Scripting Prevention Cheat Sheet
- CWE-79

5.4 Insecure Direct Object Reference — Account Access

High	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N	7.7	Open

Description

Account details are returned based solely on an account identifier supplied in the request, without any server-side check that the authenticated user is authorised to view that account. The application trusts the client-supplied identifier as an authorization decision, which it is not.

Because identifiers are sequential and predictable, a low-privileged user can simply increment or substitute the value to enumerate and read the accounts, balances and statements of every other customer. This is a Broken Object Level Authorization (BOLA) issue, consistently ranked the most prevalent and impactful class of API and web authorization flaw.

Vulnerable URL

https://demo.testfire.net/bank/showAccount?acctId=

Impact

- Cross-customer disclosure of account balances, transaction statements and personal data.
- Trivial, scriptable enumeration of the entire customer base due to sequential identifiers.
- Loss of confidentiality at scale, with direct privacy and regulatory consequences.

Steps to Reproduce

1. Log in as a standard user and note the account identifier associated with your own account.
2. Modify the request to /bank/showAccount?acctId=800003, an account you do not own.
3. Observe that the full account record for another customer is returned with a 200 response.

Proof of Concept

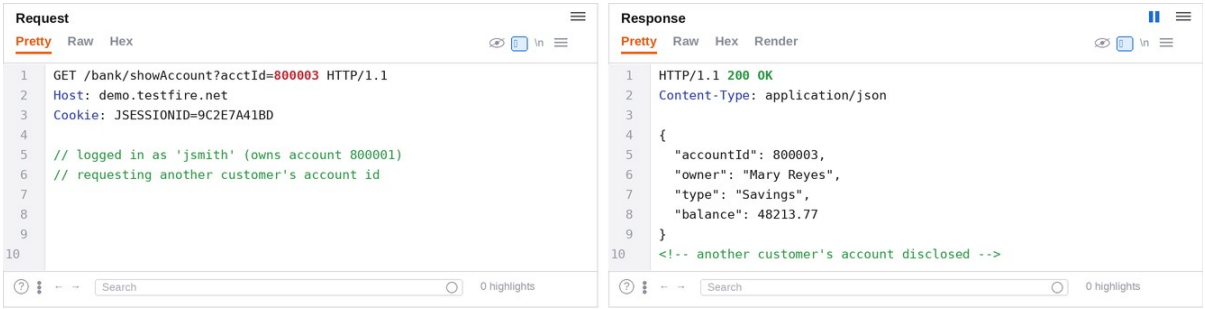

Figure 4. Burp Repeater — requesting another customer's acctId returns their full account details.

Recommendations

- Enforce object-level authorization on every request server-side, verifying that the authenticated principal owns or is entitled to the requested object before returning any data.
- Derive the account context from the authenticated session rather than from a client-supplied identifier wherever possible.

- Use unguessable identifiers (UUIDs) as defence in depth to hinder enumeration.
- Add automated authorization tests that attempt cross-tenant access, and alert on anomalous enumeration patterns.

References

- OWASP API Security Top 10 — API1:2023 BOLA
- CWE-639

5.5 Broken Authentication — No Account Lockout or Rate Limiting

High	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N	7.3	Open

Description

The authentication endpoint does not implement any anti-automation controls such as account lockout, rate limiting, exponential back-off or CAPTCHA. An attacker can therefore submit an unlimited number of credential guesses against any account.

Combined with weak or reused passwords, this makes both targeted brute-force attacks and large-scale credential-stuffing attacks practical, and there is no detective control to alert defenders that such an attack is underway.

Vulnerable URL

https://demo.testfire.net/doLogin

Impact

- Account takeover through automated guessing of weak, default or reused passwords.
- Credential-stuffing using breached username/password pairs from other services.
- No preventive or detective control limits or surfaces the attack, so compromise may go unnoticed.

Steps to Reproduce

1. Capture a failed login request to /doLogin.
2. Configure an automated password-guessing tool (for example Hydra) against the uid and passw parameters using a common-password wordlist.
3. Observe that valid credentials are recovered with no lockout, throttling or alerting.

Proof of Concept

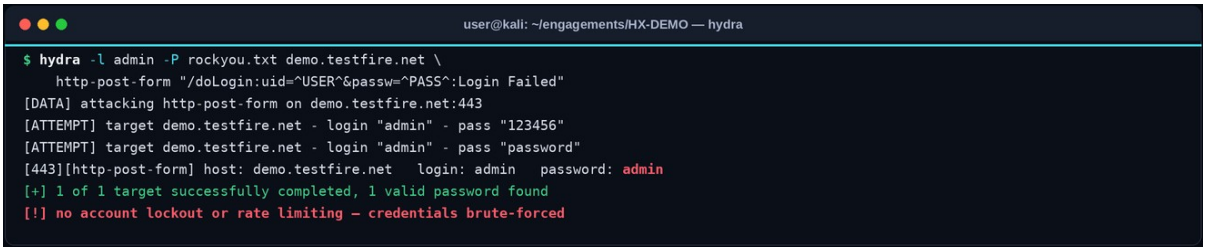

Figure 5. Hydra recovers valid credentials against the login endpoint, which applies no lockout or rate limiting.

Recommendations

- Implement progressive account lockout and per-account and per-IP rate limiting after a small number of failed attempts.
- Introduce CAPTCHA or proof-of-work after repeated failures to defeat automation.
- Enforce a strong password policy and offer/enforce multi-factor authentication for sensitive accounts.

- Monitor authentication telemetry and alert on spikes in failures, credential-stuffing signatures and impossible-travel logins.

References

- OWASP — Authentication Cheat Sheet
- CWE-307

5.6 Cross-Site Request Forgery — Funds Transfer

Medium	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N	6.5	Open

Description

The funds-transfer endpoint accepts state-changing POST requests that are authenticated only by the automatically transmitted session cookie, with no anti-CSRF token, custom header requirement or strict SameSite policy to prove the request originated from the application's own UI.

An attacker can host a page that automatically submits a transfer request; when a logged-in victim visits it, the browser attaches the victim's session cookie and the transfer executes without the victim's knowledge or consent.

Vulnerable URL

https://demo.testfire.net/bank/doTransfer

Impact

- Unauthorised transfer of funds from a victim's account to an attacker-controlled account.
- Any other state-changing action lacking a token (profile changes, payments) can be forged in the same way.
- Attacks are silent and require only that the victim visit a web page while authenticated.

Steps to Reproduce

1. Create an HTML page that auto-submits a POST to /bank/doTransfer with attacker-chosen parameters.
2. Induce a logged-in victim to open the page (for example via a link).
3. Observe the transfer complete using the victim's session, with no anti-CSRF token required.

Proof of Concept

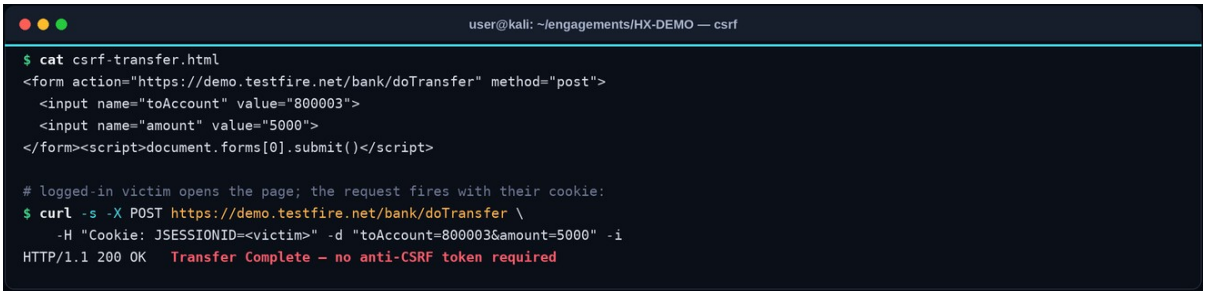

Figure 6. An auto-submitting cross-origin form completes a funds transfer; the server returns 200 OK with no anti-CSRF token.

Recommendations

- Implement anti-CSRF tokens (synchroniser-token pattern) on all state-changing requests and validate them server-side.
- Set session cookies to SameSite=Lax or Strict to prevent the browser sending them on cross-site requests.

- Require re-authentication or transaction signing for high-value actions such as transfers.
- Consider verifying the Origin/Referer header for state-changing requests as defence in depth.

References

- OWASP — CSRF Prevention Cheat Sheet
- CWE-352

5.7 Outdated JavaScript Library with Known Vulnerabilities

Medium	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N	6.2	Open

Description

The application loads an outdated jQuery library (version 1.4.2) that is affected by multiple publicly disclosed cross-site scripting and prototype-pollution vulnerabilities. Vulnerable and unmaintained third-party components are a leading cause of breaches because their flaws are public, well-documented and easily fingerprinted by automated tooling.

Even where the application's own code is sound, a vulnerable bundled library can introduce exploitable sinks and expand the overall attack surface.

Vulnerable URL

https://demo.testfire.net/

Impact

- Known XSS and prototype-pollution issues in the bundled library may be directly exploitable against users.
- Outdated components are easily identified by attackers and automated scanners, making the application an attractive target.
- Compliance frameworks frequently flag known-vulnerable components as findings in their own right.

Steps to Reproduce

1. Retrieve the application home page and inspect the included script sources.
2. Identify the reference to jquery-1.4.2.min.js in the markup.
3. Cross-reference the version against public vulnerability databases (e.g. Snyk, NVD).

Proof of Concept

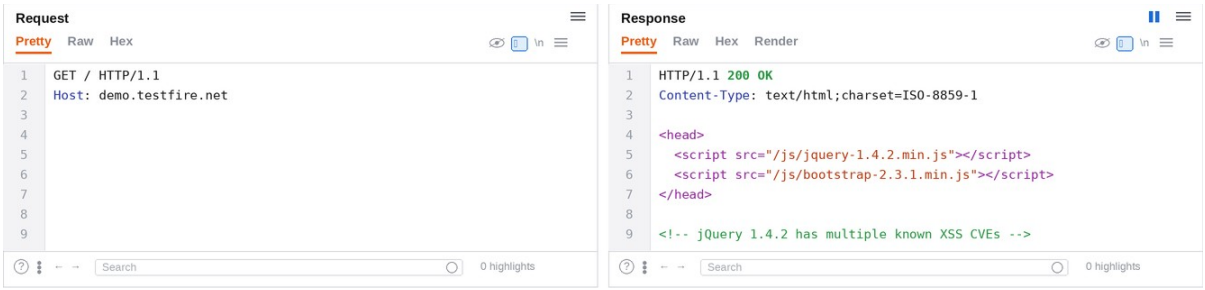

Figure 7. The page source references jQuery 1.4.2, a version affected by multiple known XSS vulnerabilities.

Recommendations

- Upgrade jQuery and all third-party libraries to current, supported versions and re-test for regressions.
- Maintain a Software Bill of Materials (SBOM) for all front-end and back-end dependencies.

- Integrate dependency scanning (for example OWASP Dependency-Check or Snyk) into the CI pipeline to fail builds on known-vulnerable components.
- Establish a patch-management process with defined SLAs for vulnerable dependencies.

References

- OWASP Top 10 — A06:2021 Vulnerable and Outdated Components
- CWE-1104

5.8 Verbose Error Messages & Stack-Trace Disclosure

Medium	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	5.3	Open

Description

When supplied with malformed input the application returns detailed Java and SQL exception messages, including fragments of the underlying query, internal class and package names, and stack-trace information. This level of detail is intended for developers and should never be exposed to end users.

The disclosed information materially assists an attacker in understanding the application's internals and in crafting precise follow-on attacks such as SQL injection, reducing the effort required to exploit other weaknesses.

Vulnerable URL

https://demo.testfire.net/bank/showAccount

Impact

- Disclosure of the technology stack, internal class structure and SQL query fragments.
- Acceleration of injection and other attacks by removing the need for blind probing.
- Potential disclosure of file paths and configuration details useful for further compromise.

Steps to Reproduce

1. Submit a single quote (') in the acctId parameter.
2. Observe a 500 response containing a SQL exception and stack trace.
3. Note the disclosed query fragment, exception type and internal class names.

Proof of Concept

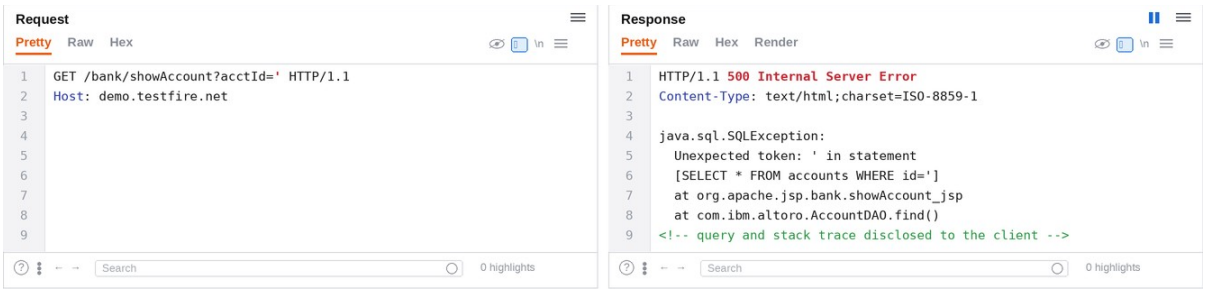

Figure 8. A malformed parameter triggers a SQL exception and stack trace that reveals the query and internal class names.

Recommendations

- Return generic, user-friendly error messages to clients and log detailed diagnostics server-side only.
- Configure a custom error handler and disable framework debug output in production environments.
- Ensure unhandled exceptions are caught centrally and never serialised to the response.
- Review logs to confirm sensitive data is not written to client-accessible locations.

References

- OWASP — Improper Error Handling
- CWE-209

5.9 Missing HTTP Security Headers

Medium	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N	5.3	Open

Description

Application responses omit several recommended HTTP security headers, including Strict-Transport-Security (HSTS), Content-Security-Policy (CSP) and X-Frame-Options. These headers instruct the browser to enforce protective behaviours and form an important layer of defence in depth.

Their absence does not create a vulnerability on its own but materially increases the likelihood and impact of other attacks such as cross-site scripting, clickjacking and protocol-downgrade man-in-the-middle attacks.

Vulnerable URL

https://demo.testfire.net/

Impact

- Increased exposure to clickjacking through the absence of X-Frame-Options or a frame-ancestors policy.
- Susceptibility to TLS-downgrade and man-in-the-middle attacks without HSTS.
- Greater impact from any cross-site scripting flaw without a restrictive Content-Security-Policy.

Steps to Reproduce

1. Issue a request to the application root and capture the response headers.
2. Compare the returned headers against the recommended set.
3. Note the absence of HSTS, CSP and X-Frame-Options.

Proof of Concept

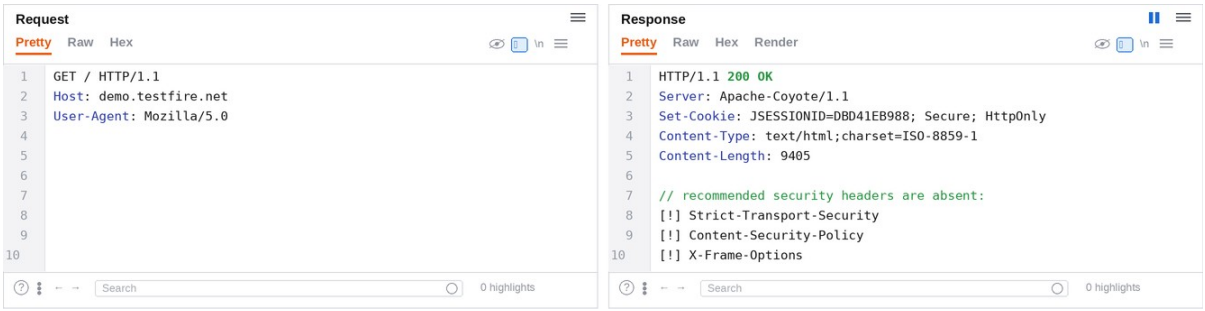

Figure 9. The response is missing HSTS, Content-Security-Policy and X-Frame-Options headers.

Recommendations

- Add Strict-Transport-Security with a long max-age and includeSubDomains, after confirming full HTTPS coverage.
- Define and deploy a restrictive Content-Security-Policy, tightening it iteratively from report-only mode.

- Set X-Frame-Options: DENY (or a CSP frame-ancestors directive) to prevent clickjacking.
- Add X-Content-Type-Options: nosniff and a sensible Referrer-Policy, and validate the result against the OWASP Secure Headers Project.

References

- OWASP Secure Headers Project
- CWE-693

5.10 Username Enumeration via Login Responses

Low	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N	3.7	Open

Description

The login endpoint returns distinguishable responses for a valid username with an incorrect password versus an entirely unknown username. This observable difference allows an attacker to determine which usernames exist on the system before mounting a targeted password attack.

While low impact in isolation, username enumeration meaningfully improves the efficiency and success rate of brute-force, credential-stuffing and phishing campaigns by removing guesswork about valid accounts.

Vulnerable URL

https://demo.testfire.net/doLogin

Impact

- Confirmation of valid usernames, improving the efficiency of subsequent authentication attacks.
- Support for targeted social-engineering and phishing against confirmed account holders.

Steps to Reproduce

1. Submit a login attempt with a known-valid username and an incorrect password and record the response.
2. Submit a login attempt with an invalid username and record the response.
3. Compare the two responses and observe that the messages differ, revealing account validity.

Proof of Concept

Figure 10. The login endpoint returns different messages for valid and invalid usernames, enabling enumeration.

Recommendations

- Return a single, generic authentication-failure message regardless of which credential is incorrect.
- Ensure response timing and HTTP status codes are identical for valid and invalid usernames.
- Apply the same anti-automation controls (rate limiting, lockout) recommended for the login endpoint.

References

- OWASP WSTG — Testing for Account Enumeration
- CWE-204

THANK YOU

HEXLOGIC